

AUDIT-09-SKILLS_MD.md

Audit & Extraction Report: Subfolder 9 of

__Lex-Novi-Æxentis-Copiæ (--📁~SKILLS.md_🔧)

Scope: Procedural skill definitions, session lifecycle management, AST structural pre-reading, and operator writing styles.

Parent Source: __Lex-Novi-Æxentis-Copiæ/--📁~SKILLS.md_🔧_
(1Cgj_AMu98K6-TNnk101W4PNwk2wEN5jE).

Status: All original files preserved 100% untouched.

1. Purpose & System Role: Procedural Repertoire (Tier 4)

This folder represents the **Procedural Memory (Tier 4)** core for both Mode A (Claude Code CLI) and Mode B (Prime Agent):

- Standardizes session closers, checkpoint handoffs, and memory safety layers.
 - Enforces the **AST Structural Pre-Read Rule**, preventing large files from saturating model context windows.
-

2. Major Technical Extractions & Discoveries

A. The AST Structural Pre-Read Pattern (AST.txt)

- *The Problem:* Standard file reading tools load entire 5,000-line files into memory, consuming up to 33,000 tokens (16% of context) just to trace a single function call, leading to context compaction.
- *The Mined Solution:*
 - Generates a 30-line method-level structural map using AST parsing (via `code_map.py` or Graphify).
 - The agent inspects the map, identifies the exact function line range, and executes scoped reads (`offset / limit`).
 - **Measured Impact:** 80% to 93% context token reduction across multi-file traces.
- *Action:* Codified into `04_skills_runtime/policies/structural_reading_policy.md` and Spec 03.

B. Clean Session Close & Memory Safety (`Close.txt` & `Happy Ending`)

- *The Problem*: Naive agents append every speculative intermediate thought into memory files, polluting the vault with reverted ideas, dead ends, or canceled instructions.
- *The Mined Solution*:
 - **Memory Safety Layer**: Classifies points by confidence; only *final* + *high-confidence* decisions enter long-term storage.
 - **Reversal Detection**: Detects undo triggers ("actually no", "take that out") so the last confirmed state wins.
 - **Memory Patching**: Emits targeted `Add/Update/Ignore` patches instead of blind-rewriting whole memory files.
 - **Session Handoff**: Prepends a 2-line summary to `SESSION_LOG.md` and formats an instant copy-paste next-session prompt.
- *Action*: Codified into `04_skills_runtime/prompt_skills/session_close_skill.md`.

C. Matt Pocock QA & Debugging Skill (`Pocock-debug.skill.txt`)

- *Content*: Interactive conversational QA session. Clarifies user issues, runs a background exploration sub-agent to learn domain language (from `UBIQUITOUS_LANGUAGE.md`), and files durable GitHub issues without stale file paths.
- *Action*: Extracted into `04_skills_runtime/prompt_skills/qa_session.md`.

D. The Technical Builder Writing Style (`SKILL.md`)

- *Content*: Practical, direct tone focused on actionable technical specifications, breaking down hardware requirements into scannable lists without unnecessary conversational fluff.
 - *Action*: Enforced across all master ecosystem outputs.
-

3. What Was Combined & What Was Trimmed

- **Fluff Trimmed**: Trimmed community rating cards and promotional web marketplace listings (`skills-for-ai`).
- **Combined Frameworks**:
 - Unified the `/close` and `Happy Ending` skills into a single canonical session lifecycle protocol in `04_skills_runtime/prompt_skills/`.
 - Integrated the AST structural reading rules with **PyGraphify** in Spec 03.